Principles of Programming Languages

Dr. Harish D. Gadade (डॉ. हरिष डी. गडदे)
Dept. of Computer Science and Engg.
COEP Technological University, Pune

Programming for Problem Solving

Sr. No.	Items	Marks
1	Mid Sem	30 Marks
2	TA (Teacher Assessment) a) Surprise Test 10 Marks b) Attendance 10 Marks c) Other if any Total Marks will be converted to 10 Marks	10 Marks
3	End Sem Exam	60 Marks
	Total	100 Marks

Total Credits = 03

To study...

- Statement Level Control Structures
- Subprograms
- Fundamentals of Subprograms
- Design Issues for Subprograms
- Local Referencing Environment
- Parameter Passing Methods

Statement Level Control Structures

- Introduction
- Selection Statements
  - Two Way Selection Statements
  - Multiple-Selection Statements
- Iterative Statements
  - Counter-Controlled Loops
  - Logically Controlled Loops
  - User-Located Loop Control Mechanism
  - Iteration Based on Data Structures

Selection Statements

- **Two Way Selection Statements**

  In Programming, programs can check the given conditions and execute the block statements accordingly.

  - If Statement
  - If Else Statements

```
if (condition) {
    // statements to execute if condition is true
}
```

```
if condition:
    # Code to be executed if the condition is True
    statement1
    statement2
    # ...
```

Selection Statements

- **Two Way Selection Statements**

In Programming, programs can check the given conditions and execute the block statements accordingly.

- If Statement
- If Else Statements

```
age = 25
if age >= 18:
    print("You are eligible to vote.")
```

```
int main() {

    int i = 10;

    // If statement
    if (i < 18) {
        printf("Eligible for vote");
    }
}
```

Selection Statements

- **Two Way Selection Statements**

In Programming, programs can check the given conditions and execute the block statements accordingly.

- If Statement
- If Else Statements

```
if (condition) {
    // Code to execute if the condition is true
} else {
    // Code to execute if the condition is false
}
```

```
if condition:
    # Code to execute if the condition is True
    # This block is indented
else:
    # Code to execute if the condition is False
    # This block is also indented
```

Selection Statements

- **Two Way Selection Statements**

In Programming, programs can check the given conditions and execute the block statements accordingly.

- If Statement
- If Else Statements

```c
int main() {
    int i = 10;

    if (i > 18) {
        printf("Eligible for vote");
    }
    else {
        printf("Not Eligible for vote");
    }
    return 0;
}
```

```python
age = 20

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Selection Statements

- **Multi Way Selection Statements**

  In Programming, programs can check the multiple conditions and execute the block statements accordingly

  - If-Else-If Statement
  - Switch-Case Statements

```
if (condition1) {
    // Code block to be executed if condition1 is true
} else if (condition2) {
    // Code block to be executed if condition1 is false and condition2 is true
} else if (condition3) {
    // Code block to be executed if condition1 and condition2 are false, and c
}
// ... (more else if blocks can be added)
else {
    // Code block to be executed if all preceding conditions are false
}
```

```
if condition1:
    # Code block to execute if condition1 is true
elif condition2:
    # Code block to execute if condition1 is false AND condition2 is true
elif condition3:
    # Code block to execute if condition1 and condition2 are false AND conditi
else:
    # Code block to execute if all preceding conditions are false
```

Selection Statements

- **Multi Way Selection Statements**

  In Programming, programs can check the multiple conditions and execute the block statements accordingly

  - If-Else-If Statement
  - Switch-Case Statements

```python
age = 20

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

```c
int main() {
    int i = 20;

    // If else ladder with three conditions
    if (i == 10)
        printf("Not Eligible");
    else if (i == 15)
        printf("wait for three years");
    else if (i == 20)
        printf("You can vote");
    else
        printf("Not a valid age");

    return 0;
}
```

Selection Statements

- **Multi Way Selection Statements**

  In Programming, programs can check the multiple conditions and execute the block statements accordingly

  - If-Else-If Statement
  - Switch-Case Statements

```
switch (expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

```
int main() {

    // variable to be used in switch statement
    int var = 18;

    // declaring switch cases
    switch (var) {
    case 15:
        printf("You are a kid");
        break;
    case 18:
        printf("Eligible for vote");
        break;
    default:
        printf("Default Case is executed");
        break;
    }

    return 0;
}
```

Statement Level Control Structures

- Iterative Statements
  - Counter-Controlled Loops
  - Logically Controlled Loops
  - User-Located Loop Control Mechanism
  - Iteration Based on Data Structures

Iterative Statements

- **Counter-Controlled Loops**
  - A counter-controlled loop is a type of loop in programming where the number of repetitions (iterations) is already known in advance.
  - The loop uses a counter (a variable) that is initialized, tested against a condition, and incremented or decremented each time until the condition becomes false.

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

Iterative Statements

- **Logically-Controlled Loops**

- A **logically controlled loop** is a type of loop where the number of repetitions is **not known in advance**. Instead, the loop continues to execute until a **logical condition** (a Boolean expression) becomes false (or true, depending on the logic).
- In short: It is controlled by a logical condition, not by a fixed counter.

```c
int main() {
    int i=0,n;
    printf("\nEnter Number : ");
    scanf("%d",&n);
    while(i<n)
    {
        printf("%d\n",i);
        i++;
    }
}
```

```c
int main() {
    int num;
    do
    {
        printf("Enter Number : ");
        scanf("%d",&num);
    }while(num<0);
}
```

- **Counter-controlled** → runs a fixed number of times.
- **Logically controlled** → runs based on a condition, which might depend on user input, sensor data, or program logic.

Iterative Statements

- **User-Located Loop Control Mechanism**

This refers to a **loop control method where the user explicitly decides when the loop should stop.**

- Unlike counter-controlled loops (fixed number of times) or logically controlled loops (stop when a condition becomes false), here the **control is placed in the user's hands.**
- The program keeps asking for input (or repeating some task) **until the user indicates to stop** (often by entering a special value, called a *sentinel*).

```c
int num;
char choice;

do {
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("You entered: %d\n", num);

    printf("Do you want to continue? (y/n): ");
    scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');
```

- The loop continues as long as the **user chooses 'y'**.
- The user is **located inside the loop** as the one deciding when to exit.

Iterative Statements

- **Iteration Based on Data Structures**

Iteration based on data structures is when a loop **automatically processes each element of a data structure** (like an array, list, string, set, dictionary, etc.) **without the programmer explicitly managing a counter or condition.**

```c
int arr[] = {10, 20, 30, 40, 50};
int n = sizeof(arr) / sizeof(arr[0]);

for (int i = 0; i < n; i++) {
    printf("%d\n", arr[i]);  // Iterating over array
}
```

```python
numbers = [10, 20, 30, 40, 50]

for num in numbers:   # Direct iteration over list
    print(num)
```

```python
student = {"name": "Ravi", "age": 20, "marks": 85}

for key, value in student.items():
    print(key, ":", value)
```

Fundamentals of Subprograms

- What is a Subprogram?
- Components of subprograms
- Types of subprograms
- Parameter Passing Methods

Fundamentals of Subprograms

What is a Subprogram?

A **subprogram** is a block of code within a program that is designed to perform a specific task. It can be invoked (called) from different parts of the program whenever that task is needed.

1. **Modularity** – They break a large program into smaller, manageable pieces.
2. **Reusability** – The same subprogram can be used multiple times in different parts of the program.
3. **Maintainability** – Easier to update or debug since logic is localized.

Fundamentals of Subprograms

- **Components of a Subprogram**
  - Name – identifier used to call the subprogram
  - Parameters – input values (optional)
  - Body – contains the code
  - Return Type – value returned (functions only)

```
int add(int a, int b)
{
        return a + b;
}
```

Fundamentals of Subprograms

- **Types of Subprograms**
  - **Procedures –** Perform an action but do not return a value.
  - **Functions –** Perform an action and return a value.

Type	Description	Returns Value?
Function	Performs computation	✅ Yes
Procedure	Performs an action (e.g., print)	❌ No

Fundamentals of Subprograms

Types of Subprograms

```python
# Function (a type of subprogram)
def square(num):
    return num * num

# Procedure (subprogram without return value)
def greet(name):
    print("Hello,", name)

# Main program
print(square(5))    # Function call
greet("Harish")     # Procedure call
```

```c
#include <stdio.h>
int square(int num)
{
    return(num*num);
}
void greet(char name[20])
{
    printf("Hello, %s\n",name);
}
int main() {
    int num=2,result;
    char name[20]="Harish";
    result=square(num);
    greet(name);
    printf("Square=%d",result);
    return 0;
}
```

Fundamentals of Subprograms

- **Parameter Passing Methods**
  - Pass by Value
  - Pass by Reference
  - Pass by Results
  - Pass by Value-Result
  - Pass by Name

Fundamentals of Subprograms

- **Pass by Value**
  - A **copy** of the actual parameter is passed.
  - Changes made inside the subprogram **do not affect** the original variable.

✅ **Safe** but uses more memory if data is large.

```c
#include <stdio.h>

void change(int x) {
    x = x + 10;   // changes local copy
    printf("Inside function: %d\n", x);
}

int main() {
    int a = 5;
    change(a);
    printf("Outside function: %d\n", a); // 'a' remains unchanged
    return 0;
}
```

Fundamentals of Subprograms

- **Pass by Reference**
  - The **address (reference)** of the variable is passed.
  - Changes inside the subprogram **directly affect** the original variable.

✅ **Efficient**, but risk of unintended modification.

```c
#include <stdio.h>

void change(int *x) {
    *x = *x + 10;   // modifies actual variable
    printf("Inside function: %d\n", *x);
}

int main() {
    int a = 5;
    change(&a);     // pass address of 'a'
    printf("Outside function: %d\n", a); // 'a' is changed
    return 0;
}
```

Fundamentals of Subprograms

- **Pass by Results**
  - Subprogram gets an **uninitialized variable**.
  - After execution, the final value is copied back to the caller.
  - In Ada, an uninitialized variable is a declared variable that has not been assigned a value.
  - Using such a variable before it is assigned a value results in undefined behavior, meaning the program's outcome is unpredictable.
  - Example: Rare, Ada

Fundamentals of Subprograms

- **Pass by Value-Result**
  - Combination of **Pass by Value** and **Pass by Result**.
  - A copy is passed **in**, and final result is copied **back out**.
  - Example: Ada, Fortran

- **Pass by Name**
  - Textual substitution: The actual expression is substituted directly into the subprogram.
  - Rare in modern languages.
  - Example: ALGOL

Fundamentals of Subprograms

- **Parameter Passing Methods**

Method	Description	Example Use
Pass by Value	Copy of actual value is passed	C, Java (primitives)
Pass by Reference	Reference to actual variable is passed	C++ (using &), Python (mutable types)
Pass by Result	Value returned to caller after execution	Rare, Ada
Pass by Value-Result	Combo of value and result	Ada, Fortran
Pass by Name	Like textual substitution	ALGOL